Quiz — Banco de Dados NoSQL (MongoDB)

65 questões objetivas — Engenharia de Software, 6º Semestre

Bloco 1 — Revisão Geral (10 questões)

Big Data, Relacional x NoSQL, ACID x BASE, CAP, Chave-Valor, Documentos, Coluna Larga, Grafos, Sharding, Replicação.

Questão 1 *[Big Data]*

Qual alternativa representa corretamente os "3 Vs" clássicos do Big Data?

A) Volume, Valor, Visualização
B) Volume, Velocidade, Variedade
C) Variabilidade, Veracidade, Valor
D) Velocidade, Volume, Vinculação

Questão 2 *[Relacional x NoSQL]*

Em um banco relacional, o schema das tabelas é definido antecipadamente. No MongoDB, por padrão, uma coleção:

A) Exige que todo documento tenha exatamente os mesmos campos, na mesma ordem
B) Não aceita dois documentos com estruturas diferentes
C) É schema-less: documentos podem ter estruturas diferentes, embora seja possível impor regras com $jsonSchema
D) Só permite alterar sua estrutura recriando o banco inteiro

Questão 3 *[ACID x BASE]*

O modelo BASE, comumente associado a bancos NoSQL distribuídos, é formado por:

A) Basically Available, Soft state, Eventually consistent
B) Atomic, Stable, Encrypted
C) Basic Access, Secure Encryption
D) Backup, Availability, Scalability, Eventual consistency

Questão 4 *[Teorema CAP]*

Segundo o Teorema CAP, em um sistema distribuído não é possível garantir simultaneamente, de forma completa:

A) Autenticação, Autorização e Auditoria
B) Consistência, Disponibilidade e Tolerância a Partição
C) Custo, Agilidade e Performance
D) Compressão, Agregação e Paralelismo

Questão 5 *[Chave-Valor]*

O Redis é um exemplo típico de banco NoSQL do modelo:

A) Orientado a grafos
B) Orientado a colunas largas
C) Chave-Valor
D) Orientado a documentos

Questão 6 *[Documentos]*

No modelo orientado a documentos (como o MongoDB), os dados são armazenados principalmente como:

A) Tabelas com linhas e colunas fixas
B) Documentos BSON/JSON agrupados em coleções

C) Grafos de nós e arestas
D) Arquivos binários sem qualquer estrutura

Questão 7 *[Coluna Larga]*

O Cassandra é classificado como um banco de dados NoSQL do modelo:

A) Documento
B) Chave-valor
C) Coluna larga (wide-column)
D) Grafo

Questão 8 *[Grafos]*

Bancos orientados a grafos, como o Neo4j, são especialmente indicados para modelar:

A) Relações complexas e altamente conectadas entre entidades, como redes sociais ou recomendação
B) Grandes volumes de logs sem qualquer relação entre si
C) Apenas dados geográficos e mapas
D) Transações bancárias exigindo apenas ACID

Questão 9 *[Sharding]*

Em MongoDB, o sharding é a técnica utilizada para:

A) Criptografar os dados armazenados
B) Distribuir (particionar) os dados entre múltiplos servidores, escalando horizontalmente
C) Fazer backup incremental automático
D) Validar o schema dos documentos antes da inserção

Questão 10 *[Replicação]*

Um Replica Set no MongoDB tem como principal objetivo:

A) Reduzir o tamanho dos documentos armazenados
B) Aumentar disponibilidade e tolerância a falhas, mantendo cópias dos dados em múltiplos nós
C) Impedir consultas concorrentes na mesma coleção
D) Converter documentos BSON em JSON puro automaticamente

Bloco 2 — CRUD e Consultas em MongoDB (35 questões)

Coleção fictícia usada nas questões: empresa_tech.funcionarios (nome, departamento, idade, matricula, salario, cargo, ativo, contato.telefone, contato.email, habilidades[], certificacoes[], endereco).

Questão 11 *[insertOne]*

Qual comando insere corretamente um único novo funcionário na coleção funcionarios?

A) db.funcionarios.insert({ nome: "Rafael Dias" })
B) db.funcionarios.insertOne({ nome: "Rafael Dias", departamento: "TI", idade: 26 })
C) db.funcionarios.insertMany({ nome: "Rafael Dias" })
D) db.funcionarios.add({ nome: "Rafael Dias" })

Questão 12 *[insertMany]*

O comando abaixo tenta inserir 3 funcionários, mas o segundo tem um _id duplicado de um documento já existente. Com a opção padrão (ordered: true), o que acontece?

```
db.funcionarios.insertMany([doc1, docComIdDuplicado, doc3])
```

A) Todos os 3 documentos são inseridos normalmente, ignorando o erro
B) Nenhum documento é inserido
C) O 1º documento é inserido, a operação para no erro do 2º, e o 3º não é inserido
D) Apenas o documento com _id duplicado é ignorado, e os outros dois são inseridos

Questão 13 *[insertMany]*

Para que o insertMany() continue inserindo os demais documentos válidos mesmo que um deles falhe (ex.: _id duplicado), deve-se usar:

A) { ordered: false }
B) { ordered: true }
C) { continueOnError: true }
D) Não é possível — insertMany() sempre para no primeiro erro

Questão 14 *[find / findOne]*

Qual a principal diferença entre find() e findOne()?

A) find() só funciona com filtros vazios
B) findOne() retorna um cursor; find() retorna um único documento
C) find() retorna um cursor (podendo iterar vários documentos); findOne() retorna apenas o primeiro documento correspondente (ou null)
D) Não há diferença, são aliases um do outro

Questão 15 *[Projeção]*

Qual comando retorna apenas os campos "nome" e "cargo" (sem o "_id") de todos os funcionários?

A) db.funcionarios.find({}, { nome: 1, cargo: 1 })
B) db.funcionarios.find({}, { nome: 1, cargo: 1, _id: 0 })
C) db.funcionarios.find({}, { nome: true, cargo: true, id: false })
D) db.funcionarios.select("nome", "cargo")

Questão 16 *[Projeção]*

Qual comando retorna todos os campos do funcionário, EXCETO "salario"?

A) db.funcionarios.find({}, { salario: 0 })
B) db.funcionarios.find({}, { salario: -1 })
C) db.funcionarios.find({}, { salario: false, outros: true })

D) db.funcionarios.find({ salario: { $exists: false } })

Questão 17 *[Operadores de comparação]*

Qual filtro retorna os funcionários com idade maior que 30?

A) db.funcionarios.find({ idade: { $gt: 30 } })
B) db.funcionarios.find({ idade: { $gte: 30 } })
C) db.funcionarios.find({ idade: > 30 })
D) db.funcionarios.find({ idade: { $lt: 30 } })

Questão 18 *[Operadores de comparação]*

Qual filtro retorna os funcionários com salário menor ou igual a 5000?

A) db.funcionarios.find({ salario: { $lt: 5000 } })
B) db.funcionarios.find({ salario: { $lte: 5000 } })
C) db.funcionarios.find({ salario: { $gte: 5000 } })
D) db.funcionarios.find({ salario: { $ne: 5000 } })

Questão 19 *[Operadores de comparação]*

Qual filtro retorna os funcionários cujo cargo NÃO é 'Estagiário'?

A) db.funcionarios.find({ cargo: { $eq: "Estagiário" } })
B) db.funcionarios.find({ cargo: { $ne: "Estagiário" } })
C) db.funcionarios.find({ cargo: "!Estagiário" })
D) db.funcionarios.find({ cargo: { $not: "Estagiário" } })

Questão 20 *[$and / $or]*

Qual filtro retorna funcionários do departamento 'TI' E com salário maior que 6000?

A) db.funcionarios.find({ $or: [{ departamento: "TI" }, { salario: { $gt: 6000 } }] })
B) db.funcionarios.find({ departamento: "TI", salario: { $gt: 6000 } })
C) db.funcionarios.find({ departamento: "TI" || salario > 6000 })
D) db.funcionarios.find({ departamento: "TI" }, { salario: { $gt: 6000 } })

Questão 21 *[$and / $or]*

Qual filtro retorna funcionários que são do departamento 'TI' OU do departamento 'Marketing'?

A) db.funcionarios.find({ departamento: "TI", departamento: "Marketing" })
B) db.funcionarios.find({ $and: [{ departamento: "TI" }, { departamento: "Marketing" }] })
C) db.funcionarios.find({ $or: [{ departamento: "TI" }, { departamento: "Marketing" }] })
D) db.funcionarios.find({ departamento: ["TI", "Marketing"] })

Questão 22 *[$and / $or combinados]*

Qual filtro retorna funcionários ativos E que sejam do departamento 'TI' OU tenham mais de 40 anos?

A) db.funcionarios.find({ ativo: true, $or: [{ departamento: "TI" }, { idade: { $gt: 40 } }] })
B) db.funcionarios.find({ $or: [{ ativo: true, departamento: "TI" }, { idade: { $gt: 40 } }] })
C) db.funcionarios.find({ ativo: true, departamento: "TI", idade: { $gt: 40 } })
D) db.funcionarios.find({ $and: { ativo: true, departamento: "TI" } })

Questão 23 *[$in / $nin]*

Qual filtro retorna funcionários cujo cargo é 'Desenvolvedora', 'Analista' ou 'Gerente'?

A) db.funcionarios.find({ cargo: { $all: ["Desenvolvedora", "Analista", "Gerente"] } })
B) db.funcionarios.find({ cargo: { $in: ["Desenvolvedora", "Analista", "Gerente"] } })
C) db.funcionarios.find({ cargo: { $or: ["Desenvolvedora", "Analista", "Gerente"] } })

D) db.funcionarios.find({ cargo: ["Desenvolvedora", "Analista", "Gerente"] })

Questão 24 *[$in / $nin]*

Qual filtro retorna funcionários cujo departamento NÃO é 'RH' nem 'Financeiro'?

A) db.funcionarios.find({ departamento: { $nin: ["RH", "Financeiro"] } })
B) db.funcionarios.find({ departamento: { $ne: ["RH", "Financeiro"] } })
C) db.funcionarios.find({ departamento: { $not: ["RH", "Financeiro"] } })
D) db.funcionarios.find({ departamento: { $nin: "RH, Financeiro" } })

Questão 25 *[Documento embutido]*

Qual filtro retorna funcionários cujo e-mail de contato é 'fernanda.rocha@techcorp.com'?

A) db.funcionarios.find({ email: "fernanda.rocha@techcorp.com" })
B) db.funcionarios.find({ "contato.email": "fernanda.rocha@techcorp.com" })
C) db.funcionarios.find({ contato: { email: "fernanda.rocha@techcorp.com" } })
D) db.funcionarios.find({ contato: "fernanda.rocha@techcorp.com" })

Questão 26 *[Documento embutido]*

Qual filtro retorna funcionários que moram na cidade 'São Paulo' (campo endereco.cidade)?

A) db.funcionarios.find({ "endereco.cidade": "São Paulo" })
B) db.funcionarios.find({ endereco: "São Paulo" })
C) db.funcionarios.find({ cidade: "São Paulo" })
D) db.funcionarios.find({ "endereco->cidade": "São Paulo" })

Questão 27 *[Consulta em array]*

Um funcionário tem habilidades: ["MongoDB", "Node.js", "Docker"]. Qual filtro encontra funcionários que possuem "Docker" entre suas habilidades?

A) db.funcionarios.find({ habilidades: "Docker" })
B) db.funcionarios.find({ habilidades: ["Docker"] })
C) db.funcionarios.find({ habilidades: { $eq: ["Docker"] } })
D) db.funcionarios.find({ "habilidades[0]": "Docker" })

Questão 28 *[$all]*

Qual filtro encontra funcionários que possuem TODAS as habilidades "MongoDB" e "Docker" (podendo ter outras também)?

A) db.funcionarios.find({ habilidades: { $all: ["MongoDB", "Docker"] } })
B) db.funcionarios.find({ habilidades: { $in: ["MongoDB", "Docker"] } })
C) db.funcionarios.find({ habilidades: ["MongoDB", "Docker"] })
D) db.funcionarios.find({ habilidades: { $and: ["MongoDB", "Docker"] } })

Questão 29 *[$size]*

Qual filtro retorna funcionários que possuem exatamente 3 habilidades cadastradas?

A) db.funcionarios.find({ habilidades: { $length: 3 } })
B) db.funcionarios.find({ habilidades: { $size: 3 } })
C) db.funcionarios.find({ "habilidades.length": 3 })
D) db.funcionarios.find({ habilidades: { $count: 3 } })

Questão 30 *[Array de documentos embutidos]*

Cada funcionário tem certificacoes: [{ nome: "AWS Certified", ano: 2023 }, { nome: "MongoDB Associate", ano: 2024 }]. Qual filtro garante encontrar quem tem uma certificação "MongoDB Associate" obtida em 2024 (mesmo elemento do array)?

A) db.funcionarios.find({ "certificacoes.nome": "MongoDB Associate", "certificacoes.ano": 2024 })
B) db.funcionarios.find({ certificacoes: { $elemMatch: { nome: "MongoDB Associate", ano: 2024 } } })
C) db.funcionarios.find({ certificacoes: { nome: "MongoDB Associate", ano: 2024 } })
D) db.funcionarios.find({ "certificacoes.0.nome": "MongoDB Associate" })

Questão 31 *[Array de documentos embutidos]*

Qual filtro encontra funcionários cujo PRIMEIRO item do array certificacoes tem o nome 'AWS Certified'?

A) db.funcionarios.find({ "certificacoes.0.nome": "AWS Certified" })
B) db.funcionarios.find({ "certificacoes[0].nome": "AWS Certified" })
C) db.funcionarios.find({ certificacoes: { $first: "AWS Certified" } })
D) db.funcionarios.find({ "certificacoes.first.nome": "AWS Certified" })

Questão 32 *[updateOne]*

Qual comando atualiza o salário do funcionário com _id igual a 1 para 7200?

A) db.funcionarios.updateOne({ _id: 1 }, { salario: 7200 })
B) db.funcionarios.updateOne({ _id: 1 }, { $set: { salario: 7200 } })
C) db.funcionarios.updateMany({ _id: 1 }, { $set: { salario: 7200 } })
D) db.funcionarios.update({ salario: 7200 }, { _id: 1 })

Questão 33 *[updateMany]*

Qual comando marca como inativos (ativo: false) TODOS os funcionários do departamento 'Financeiro'?

A) db.funcionarios.updateOne({ departamento: "Financeiro" }, { $set: { ativo: false } })
B) db.funcionarios.updateMany({ departamento: "Financeiro" }, { $set: { ativo: false } })
C) db.funcionarios.updateMany({ ativo: false }, { $set: { departamento: "Financeiro" } })
D) db.funcionarios.replaceMany({ departamento: "Financeiro" }, { ativo: false })

Questão 34 *[replaceOne]*

Qual a principal diferença entre replaceOne() e updateOne() com $set?

A) replaceOne() substitui o documento inteiro (exceto _id); updateOne()+$set altera apenas os campos informados, preservando os demais
B) Não há diferença nenhuma entre os dois
C) replaceOne() só funciona em coleções sem validação de schema
D) updateOne()+$set sempre apaga o documento antes de recriá-lo

Questão 35 *[deleteOne]*

Qual comando remove apenas UM funcionário cujo nome é 'Rafael Dias' (mesmo que existam duplicados)?

A) db.funcionarios.deleteMany({ nome: "Rafael Dias" })
B) db.funcionarios.deleteOne({ nome: "Rafael Dias" })
C) db.funcionarios.remove({ nome: "Rafael Dias" }, true)
D) db.funcionarios.drop({ nome: "Rafael Dias" })

Questão 36 *[deleteMany]*

Qual comando remove TODOS os funcionários inativos (ativo: false)?

A) db.funcionarios.deleteOne({ ativo: false })
B) db.funcionarios.deleteMany({ ativo: false })
C) db.funcionarios.drop({ ativo: false })

D) db.funcionarios.remove()

Questão 37 *[$push]*

Qual comando adiciona a habilidade "Kubernetes" ao array habilidades do funcionário com _id 1?

A) db.funcionarios.updateOne({ _id: 1 }, { $push: { habilidades: "Kubernetes" } })
B) db.funcionarios.updateOne({ _id: 1 }, { $set: { habilidades: "Kubernetes" } })
C) db.funcionarios.updateOne({ _id: 1 }, { $add: { habilidades: "Kubernetes" } })
D) db.funcionarios.updateOne({ _id: 1 }, { $append: { habilidades: "Kubernetes" } })

Questão 38 *[$push com $each e $position]*

Qual comando insere as habilidades "Kafka" e "Redis" nas DUAS PRIMEIRAS posições (posição 0) do array habilidades?

A) db.funcionarios.updateOne({ _id: 1 }, { $push: { habilidades: ["Kafka", "Redis"] } })
B) db.funcionarios.updateOne({ _id: 1 }, { $push: { habilidades: { $each: ["Kafka", "Redis"], $position: 0 } } })
C) db.funcionarios.updateOne({ _id: 1 }, { $push: { habilidades: { $each: ["Kafka", "Redis"] } } })
D) db.funcionarios.updateOne({ _id: 1 }, { $set: { "habilidades.0": "Kafka", "habilidades.1": "Redis" } })

Questão 39 *[$addToSet x $push]*

Qual a diferença entre $addToSet e $push ao adicionar "Docker" a um array habilidades que já contém "Docker"?

A) Não há diferença — ambos sempre adicionam o valor
B) $addToSet só adiciona o valor se ele ainda não existir no array; $push sempre adiciona, podendo gerar duplicados
C) $push só adiciona se o valor não existir; $addToSet sempre adiciona
D) $addToSet remove valores duplicados existentes automaticamente

Questão 40 *[$pull]*

Qual comando remove a habilidade "Docker" do array habilidades de um funcionário?

A) db.funcionarios.updateOne({ _id: 1 }, { $remove: { habilidades: "Docker" } })
B) db.funcionarios.updateOne({ _id: 1 }, { $pop: { habilidades: "Docker" } })
C) db.funcionarios.updateOne({ _id: 1 }, { $pull: { habilidades: "Docker" } })
D) db.funcionarios.updateOne({ _id: 1 }, { $set: { habilidades: { $ne: "Docker" } } })

Questão 41 *[$pop]*

Qual comando remove o ÚLTIMO elemento do array habilidades de um funcionário?

A) db.funcionarios.updateOne({ _id: 1 }, { $pop: { habilidades: 1 } })
B) db.funcionarios.updateOne({ _id: 1 }, { $pop: { habilidades: -1 } })
C) db.funcionarios.updateOne({ _id: 1 }, { $pull: { habilidades: -1 } })
D) db.funcionarios.updateOne({ _id: 1 }, { $pop: { habilidades: "last" } })

Questão 42 *[Operador posicional $]*

Qual comando atualiza o campo "ano" para 2025 no elemento do array certificacoes cujo nome é "AWS Certified", sem afetar os demais elementos?

A) db.funcionarios.updateOne({ "certificacoes.nome": "AWS Certified" }, { $set: { "certificacoes.ano": 2025 } })
B) db.funcionarios.updateOne({ "certificacoes.nome": "AWS Certified" }, { $set: { "certificacoes.$.ano": 2025 } })
C) db.funcionarios.updateMany({}, { $set: { "certificacoes.$.ano": 2025 } })
D) db.funcionarios.updateOne({ certificacoes: "AWS Certified" }, { $set: { ano: 2025 } })

Questão 43 *[arrayFilters]*

Uma funcionária tem duas certificações. Qual comando atualiza o "ano" para 2025 APENAS na certificação cujo nome é "MongoDB Associate", usando arrayFilters?

A) db.funcionarios.updateOne({ _id: 1 }, { $set: { "certificacoes.$[cert].ano": 2025 } }, { arrayFilters: [{ "cert.nome": "MongoDB Associate" }] })

B) db.funcionarios.updateOne({ _id: 1 }, { $set: { "certificacoes.$.ano": 2025 } })

C) db.funcionarios.updateMany({ "certificacoes.nome": "MongoDB Associate" }, { $set: { ano: 2025 } })

D) db.funcionarios.updateOne({ _id: 1 }, { $set: { certificacoes: { ano: 2025 } } })

Questão 44 *[Embedding (documento dentro de documento)]*

Qual comando adiciona um subdocumento endereco: { cidade: "Curitiba", uf: "PR" } a um funcionário que ainda não possui esse campo?

A) db.funcionarios.updateOne({ _id: 2 }, { $set: { endereco: { cidade: "Curitiba", uf: "PR" } } })

B) db.funcionarios.updateOne({ _id: 2 }, { $push: { endereco: { cidade: "Curitiba", uf: "PR" } } })

C) db.funcionarios.updateOne({ _id: 2 }, { $embed: { endereco: { cidade: "Curitiba", uf: "PR" } } })

D) db.funcionarios.insertOne({ endereco: { cidade: "Curitiba", uf: "PR" } })

Questão 45 *[Consulta integradora]*

Qual comando retorna os funcionários ATIVOS, do departamento 'TI', com salário acima de 6000, ordenados do MAIOR para o MENOR salário?

A) db.funcionarios.find({ ativo: true, departamento: "TI", salario: { $gt: 6000 } }).sort({ salario: 1 })

B) db.funcionarios.find({ ativo: true, departamento: "TI", salario: { $gt: 6000 } }).sort({ salario: -1 })

C) db.funcionarios.find({ $or: [{ ativo: true }, { departamento: "TI" }, { salario: { $gt: 6000 } }] }).sort({ salario: -1 })

D) db.funcionarios.find({ ativo: true, departamento: "TI", salario: { $gt: 6000 } }).orderBy("salario", "desc")

Bloco 3 — Regex em MongoDB (20 questões)

Metacaracteres, âncoras (^ $), classes de caracteres ([abc]/[^abc]), alternância (|), quantificadores (+ ? {n} {n,} {n,m}), \d \w \s, case sensitivity e $elemMatch + regex.*

Questão 46 *[Regex — Metacaracteres]*

O que caracteriza um "metacaractere" dentro de uma expressão regular (regex)?

A) É qualquer letra maiúscula usada no padrão de busca

B) É um símbolo que tem um significado especial na regex (como ^, $, ., *), em vez de representar apenas ele mesmo

C) É um caractere que só pode ser usado em bancos relacionais

D) É a última letra, obrigatória em todo padrão de regex

Questão 47 *[Regex — Início de string (^)]*

Qual filtro encontra funcionários cujo e-mail COMEÇA com "fernanda"?

A) db.funcionarios.find({ "contato.email": { $regex: "fernanda" } })

B) db.funcionarios.find({ "contato.email": { $regex: "^fernanda" } })

C) db.funcionarios.find({ "contato.email": { $regex: "fernanda$" } })

D) db.funcionarios.find({ "contato.email": { $startsWith: "fernanda" } })

Questão 48 *[Regex — Fim de string ($)]*

Qual filtro encontra funcionários cujo e-mail TERMINA com "@techcorp.com"?

A) db.funcionarios.find({ "contato.email": { $regex: "@techcorp.com$" } })

B) db.funcionarios.find({ "contato.email": { $regex: "^@techcorp.com" } })

C) db.funcionarios.find({ "contato.email": { $regex: "@techcorp.com" } })

D) db.funcionarios.find({ "contato.email": { $endsWith: "@techcorp.com" } })

Questão 49 *[Regex — Um caractere qualquer (.)]*

Na sintaxe de expressões regulares, qual metacaractere representa "qualquer caractere" (exceto quebra de linha) em uma determinada posição?

A) \d

B) .

C) *

D) \w

Questão 50 *[Regex — Caractere de uma lista [ ]]*

Qual regex encontra funcionários cuja matrícula COMEÇA com "F" ou "T" (ex.: "F1023" ou "T3301")?

A) db.funcionarios.find({ matricula: { $regex: "^[FT]" } })

B) db.funcionarios.find({ matricula: { $regex: "^FT" } })

C) db.funcionarios.find({ matricula: { $regex: "[^FT]" } })

D) db.funcionarios.find({ matricula: { $in: "^[FT]" } })

Questão 51 *[Regex — Caractere fora da lista [^]]*

Qual regex encontra funcionários cuja matrícula NÃO começa nem com "F" nem com "T"?

A) db.funcionarios.find({ matricula: { $regex: "^[FT]" } })

B) db.funcionarios.find({ matricula: { $regex: "^[^FT]" } })

C) db.funcionarios.find({ matricula: { $regex: "^[!FT]" } })

D) db.funcionarios.find({ matricula: { $not: "^[FT]" } })

Questão 52 *[Regex — Alternância (OU) |]*

Qual regex encontra funcionários cujo cargo é "Analista" OU "Assistente", usando um único padrão de regex (sem $in)?

A) db.funcionarios.find({ cargo: { $regex: "Analista|Assistente" } })
B) db.funcionarios.find({ cargo: { $regex: "Analista&Assistente" } })
C) db.funcionarios.find({ cargo: { $regex: "Analista,Assistente" } })
D) db.funcionarios.find({ cargo: { $regex: "Analista+Assistente" } })

Questão 53 *[Regex — 0 ou mais vezes (*)]*

O quantificador * (asterisco), aplicado logo após um caractere ou grupo em uma regex, significa que esse elemento deve aparecer:

A) Exatamente uma vez
B) Zero ou mais vezes
C) Uma ou mais vezes
D) No máximo uma vez

Questão 54 *[Regex — 1 ou mais vezes (+)]*

Qual regex garante que o e-mail tenha PELO MENOS um dígito logo antes do "@" (ex.: "ana1@techcorp.com")?

A) db.funcionarios.find({ "contato.email": { $regex: "\d+@" } })
B) db.funcionarios.find({ "contato.email": { $regex: "\d*@" } })
C) db.funcionarios.find({ "contato.email": { $regex: "\d?@" } })
D) db.funcionarios.find({ "contato.email": { $regex: "\d{0}@" } })

Questão 55 *[Regex — 0 ou 1 vez / opcional (?)]*

Na regex "colou?r", o "u?" logo após o "colo" significa que a letra "u" é:

A) Obrigatória e deve aparecer exatamente uma vez
B) Opcional — pode aparecer zero ou uma vez (o padrão casaria com "color" e com "colour")
C) Proibida nessa posição
D) Repetida indefinidamente

Questão 56 *[Regex — Um dígito \d / [0-9]]*

Qual regex encontra funcionários cujo telefone contém APENAS dígitos (ex.: "11988887777"), sem espaços, parênteses ou traços?

A) db.funcionarios.find({ "contato.telefone": { $regex: "^[0-9]+$" } })
B) db.funcionarios.find({ "contato.telefone": { $regex: "^[0-9]$" } })
C) db.funcionarios.find({ "contato.telefone": { $regex: "[a-z]+" } })
D) db.funcionarios.find({ "contato.telefone": { $regex: "^\w$" } })

Questão 57 *[Regex — Letra, número ou _ (\w)]*

O metacaractere \w, em uma expressão regular, representa:

A) Apenas espaços em branco
B) Qualquer letra, número ou underscore (_) — equivalente a [A-Za-z0-9_]
C) Apenas caracteres de pontuação
D) Qualquer caractere, sem nenhuma exceção

Questão 58 *[Regex — Espaço em branco (\s)]*

Qual regex encontra funcionários cujo cargo tem EXATAMENTE duas palavras separadas por um espaço (ex.: "Analista Pleno"), usando \s?

A) db.funcionarios.find({ cargo: { $regex: "^\w+\s\w+$" } })
B) db.funcionarios.find({ cargo: { $regex: "^\w+\w+$" } })

C) db.funcionarios.find({ cargo: { $regex: "^\d+\s\d+$" } })
D) db.funcionarios.find({ cargo: { $regex: "\s" } })

Questão 59 *[Regex — Quantidade exata {n}]*

Qual regex garante que a matrícula tenha EXATAMENTE 4 dígitos logo após a letra inicial (ex.: "F1023")?

A) db.funcionarios.find({ matricula: { $regex: "^[A-Z]\d{4}$" } })
B) db.funcionarios.find({ matricula: { $regex: "^[A-Z]\d{4,}$" } })
C) db.funcionarios.find({ matricula: { $regex: "^[A-Z]\d{0,4}$" } })
D) db.funcionarios.find({ matricula: { $regex: "^[A-Z]\d*$" } })

Questão 60 *[Regex — {n,} pelo menos n vezes]*

O quantificador {3,} em uma regex significa que o elemento anterior deve se repetir:

A) Exatamente 3 vezes
B) No máximo 3 vezes
C) 3 vezes ou mais, sem limite superior
D) Entre 0 e 3 vezes

Questão 61 *[Regex — {n,m} entre n e m vezes]*

Qual regex encontra valores de um campo hipotético "codigoAcesso" com ENTRE 6 e 10 caracteres alfanuméricos (\w)?

A) db.funcionarios.find({ codigoAcesso: { $regex: "^\w{6,10}$" } })
B) db.funcionarios.find({ codigoAcesso: { $regex: "^\w{6}$" } })
C) db.funcionarios.find({ codigoAcesso: { $regex: "^\w{10,6}$" } })
D) db.funcionarios.find({ codigoAcesso: { $regex: "^\w+{6,10}$" } })

Questão 62 *[Regex — escapando caracteres especiais]*

No padrão de regex "@techcorp.com$", o ponto (.) antes de "com" é tecnicamente interpretado como:

A) Um ponto literal, sempre
B) Qualquer caractere (é um metacaractere de regex) — para buscar um ponto literal, o correto seria escapá-lo como \.
C) Um erro de sintaxe
D) O fim obrigatório da string

Questão 63 *[Regex — Case sensitive x insensitive (i)]*

Qual filtro encontra funcionários cujo nome CONTÉM a palavra "Silva" em qualquer posição, ignorando maiúsculas/minúsculas?

A) db.funcionarios.find({ nome: { $regex: "Silva" } })
B) db.funcionarios.find({ nome: { $regex: "Silva", $options: "i" } })
C) db.funcionarios.find({ nome: "Silva" })
D) db.funcionarios.find({ nome: { $like: "%Silva%" } })

Questão 64 *[Regex + $not]*

Qual filtro encontra funcionários cujo nome NÃO contém a palavra "Silva" (em qualquer caixa)?

A) db.funcionarios.find({ nome: { $regex: "Silva", $options: "i" } })
B) db.funcionarios.find({ nome: { $not: { $regex: "Silva", $options: "i" } } })
C) db.funcionarios.find({ nome: { $ne: /Silva/i } })
D) db.funcionarios.find({ nome: { $nin: "Silva" } })

Questão 65 *[Regex + $elemMatch]*

Qual filtro encontra funcionários que possuem, no array certificacoes, algum item cujo "nome" contém (via regex, ignorando maiúsculas/minúsculas) a palavra "aws"?

A) db.funcionarios.find({ "certificacoes.nome": { $regex: "aws", $options: "i" } })

B) db.funcionarios.find({ certificacoes: { $regex: "aws", $options: "i" } })

C) db.funcionarios.find({ certificacoes.nome: "aws" })

D) db.funcionarios.find({ certificacoes: { $elemMatch: { nome: /aws/ } } })